

Proposal: Detached Local Storage Pattern

Attaching Profile Declarations to Filesystem Content and Downloads

1. Introduction to the Problem Space

In modern data ecosystems (such as BIM, GIS, and Open Science), data must be highly diverse and rapidly evolvable. To achieve interoperability without central monopolization, different communities require specialized “profiles” (sub-schemas, validation shapes, and localized compliance rules) built on top of base formats.

On the Web, managing this diversity is low-cost and dynamic thanks to Content Negotiation and HTTP headers. However, **as soon as a data resource lands on a local filesystem, this rich contextual metadata is instantly lost.** Users and applications are forced to rely entirely on ambiguous file extensions (e.g., `.xml`, `.json`, `.ifc`, `.gpkg`). These extensions only specify the syntax, completely hiding the specific profile or semantic namespace in use.

To bridge this gap, we need a zero-cost, radical-transparency pattern to bootstrap profile compliance directly on physical filesystems, ensuring that files remain self-describing, discoverable, and linkable without modifying the original payload binaries.

2. Inspiration

This pattern borrows heavily from proven web standards and established filesystem conventions:

- * **The Web:** The use of HTTP `Link` headers with `rel="profile"` (RFC 6906) and the decoupling of resource locations from their architectural constraints.
- * **Deterministic Sidecars:** The decades-old IT convention of using auxiliary files for low-level metadata bootstrapping—such as `package.tar.gz.sha256` for integrity, `video.mp4.srt` for subtitles, or `texture.png.meta` in modern game engines.
- * **Zero-Invasive Extension:** Leaving the original payload untouched so that legacy, profile-unaware applications can read the files without breaking.

3. Core References

- **RFC 6906:** The ‘profile’ Link Relation Type.
- **RFC 9264:** Linkset: Media Types and Link Relations (Representing links between resources as JSON/JSON-LD).
- **W3C Profiles Vocabulary:** Content Negotiation by Profile.

4. The Core Pattern

The Radical Transparency Pattern operates on two distinct, synchronized layers:

4.1 Tier A: The Web Layer (Download & Transport)

When a client requests a resource, the server **MUST** emit the profile declarations. It does this via standard HTTP Link headers pointing to the applicable profile URIs, or by referencing an external linkset document:

```
HTTP/1.1 200 OK
Content-Type: application/x-step
Link: <https://example.org/profiles/nl-sfb-v2>; rel="profile",
      <file:///./building.ifc.ls.json>; rel="linkset"
```

4.2 Tier B: The Filesystem Layer (At Rest)

Once downloaded, the HTTP header context is materialized on the local file system using two mutually reinforcing mechanisms:

4.2.1 The Deterministic Sidecar (*.ls.json) For any payload file named [filename].[ext], an optional sidecar file named [filename].[ext].ls.json is placed in the exact same directory. * The sidecar **MUST** be a valid **RFC 9264 Linkset** serialized as JSON or JSON-LD. * It explicitly binds the local payload file to its global, resolvable profile URIs.

```
{
  "linkset": [
    {
      "anchor": "file:///./building.ifc",
      "profile": [
        {"href": "https://standards.buildingsmart.org/IFC/RELEASE/IFC4/ADD2/TC1/HTML/"},
        {"href": "https://bimloket.nl/profile/nl-sfb-v2"}
      ],
      "type": [
        {"href": "application/x-step"}
      ]
    }
  ]
}
```

4.2.2 The OS-Level Accelerator (xattr) To allow blazingly fast indexing by the operating system or localized daemons without parsing the filesystem directory tree, compliant tools can optionally write an Extended File Attribute to the payload file: * **Attribute Name:** user.linkset * **Attribute Value:** A valid URI string. This can either point to the local sidecar (file:///./building.ifc.ls.json) or point directly to an authoritative,

live web resource (<https://api.example.org/models/123/linkset>) if online validation is preferred.

Fallback Strategy: If the file is transferred over an environment that strips extended attributes (e.g., FAT32 drives, email attachments), the architecture gracefully falls back to looking for the deterministic `*.ls.json` filename convention.

5. Application Examples

5.1 BIM Industry (IFC File)

A construction engineer downloads a structural model: * **Payload:** `bridge_design.ifc` (Legacy BIM software opens this normally to view the 3D geometry). * **Sidecar:** `bridge_design.ifc.ls.json` (A compliance checker reads this sidecar, detects that the file claims conformity to the <https://bimloket.nl/profile/ifc4-infra> profile, and automatically executes the correct SHACL validation rules against it).

5.2 GIS Industry (GeoPackage File)

A city planner exports a parcel dataset: * **Payload:** `amsterdam_parcel.gpkg` (A standard SQLite container). * **Extended Attribute:** `user.linkset` → <https://data.amsterdam.nl/datasets/parcels/profile-linkset.json> * **Result:** A GIS application immediately knows which specific municipal metadata standards and coordinate reference profile transformations apply, without having to extract and parse internal `gpkg_metadata` system tables first.